

1. Particle Motion Simulator

Description:

Simulate the motion of particles in a two-dimensional space under the influence of forces.

Specifications:

- Structure: Represents particle properties (mass, position, velocity).
- Array: Stores the position and velocity vectors of multiple particles.
- Union: Handles force types (gravitational, electric, or magnetic).
- Strings: Define force types applied to particles.
- const Pointers: Protect particle properties.
- Double Pointers: Dynamically allocate memory for the particle system.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <math.h>
```

```
struct Particle
```

```
{
```

```
    const int *particleID;
```

```
    float mass;
```

```
    float position[2];
```

```
    float velocity[2];
```

```
    union
```

```
    {
```

```
        float gravitational;
```

```
        float electric;
```

```
        float magnetic;
```

```

    } force;

    int forceType; // 1 for Gravitational, 2 for Electric, 3 for Magnetic
};

void addParticle(struct Particle **particles, int *particleCount);
void displayParticleDetails(struct Particle *particles, int particleCount);
void simulateMotion(struct Particle *particles, int particleCount, float timeStep);

int main()
{
    struct Particle *particles = NULL;
    int particleCount = 0, option;
    do
    {
        printf("\nParticle Motion Simulator\n");
        printf("1. Add Particle\n");
        printf("2. Display Particle Details\n");
        printf("3. Simulate Motion\n");
        printf("4. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addParticle(&particles, &particleCount);
                    break;
            case 2: displayParticleDetails(particles, particleCount);
                    break;

```

```

    case 3: if (particleCount == 0)
        printf("No particles available for simulation\n");
    else
    {
        float timeStep;
        printf("Enter the time step for simulation: ");
        scanf("%f", &timeStep);
        simulateMotion(particles, particleCount, timeStep);
    }
    break;

    case 4: for (int i = 0; i < particleCount; i++)
        free((int *)particles[i].particleID);
        free(particles);
        printf("Exiting the simulator\n");
        break;

    default: printf("Invalid option\n");
}
} while (option != 4);
return 0;
}

```

```

void addParticle(struct Particle **particles, int *particleCount)
{
    int tempParticleID;
    struct Particle newParticle;
    printf("Enter Particle ID: ");
    scanf("%d", &tempParticleID);
}

```

[illegible]

```

(*particles)[*particleCount] = newParticle;
(*particleCount)++;
printf("Particle added successfully\n");
}

```

```

void displayParticleDetails(struct Particle *particles, int particleCount)
{
    if (particleCount == 0)
    {
        printf("No particle details available\n");
        return;
    }
    printf("\nParticle Details\n");
    for (int i = 0; i < particleCount; i++)
    {
        printf("Particle ID: %d\n", *(particles[i].particleID));
        printf("Mass: %.2f\n", particles[i].mass);
        printf("Position: (%.2f, %.2f)\n", particles[i].position[0],
                                                    particles[i].position[1]);
        printf("Velocity: (%.2f, %.2f)\n", particles[i].velocity[0],
                                                    particles[i].velocity[1]);
        switch (particles[i].forceType)
        {
            case 1: printf("Force Type: Gravitational\n");
                    printf("Force Magnitude: %.2f\n", particles[i].force.gravitational);
                    break;
            case 2: printf("Force Type: Electric\n");

```

```

        printf("Force Magnitude: %.2f\n", particles[i].force.electric);
        break;
    case 3: printf("Force Type: Magnetic\n");
        printf("Force Magnitude: %.2f\n", particles[i].force.magnetic);
        break;
    default: printf("Invalid force type\n");
}
}
}

```

```

void simulateMotion(struct Particle *particles, int particleCount, float timeStep)
{
    printf("\nSimulating motion for %d particles with time step %.2f\n",
        particleCount, timeStep);
    for (int i = 0; i < particleCount; i++)
    {
        float forceMagnitude;
        switch (particles[i].forceType)
        {
            case 1: forceMagnitude = particles[i].force.gravitational;
                break;
            case 2: forceMagnitude = particles[i].force.electric;
                break;
            case 3: forceMagnitude = particles[i].force.magnetic;
                break;
            default: printf("Invalid force type for Particle ID: %d\n",
                *(particles[i].particleID));

```

```

        continue;
    }
    float acceleration = forceMagnitude / particles[i].mass;
    particles[i].velocity[0] += acceleration * timeStep;
    particles[i].velocity[1] += acceleration * timeStep;
    particles[i].position[0] += particles[i].velocity[0] * timeStep;
    particles[i].position[1] += particles[i].velocity[1] * timeStep;
    printf("Particle ID: %d\n", *(particles[i].particleID));
    printf("New Position: (%.2f, %.2f)\n", particles[i].position[0],
                                                particles[i].position[1]);
    printf("New Velocity: (%.2f, %.2f)\n", particles[i].velocity[0],
                                                particles[i].velocity[1]);
}
}

```

2. Electromagnetic Field Calculator

Description:

Calculate the electromagnetic field intensity at various points in space.

Specifications:

- Structure: Stores field parameters (electric field, magnetic field, and position).
- Array: Holds field values at discrete points.
- Union: Represents either electric or magnetic field components.
- Strings: Represent coordinate systems (Cartesian, cylindrical, spherical).
- const Pointers: Prevent modification of field parameters.
- Double Pointers: Manage memory for field grid allocation dynamically.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
```

```
struct FieldPoint
{
    const int *pointID;
    float position[3];
    union
    {
        float electricField[3];
        float magneticField[3];
    } field;
    int fieldType;    // 1 for Electric, 2 for Magnetic
    char coordinateSystem[20]; // Cartesian, Cylindrical, Spherical
};
```

```
void addFieldPoint(struct FieldPoint **fieldPoints, int *pointCount);
void displayFieldPoints(struct FieldPoint *fieldPoints, int pointCount);
void calculateFieldIntensity(struct FieldPoint *fieldPoints, int pointCount);
```

```
int main()
{
    struct FieldPoint *fieldPoints = NULL;
    int pointCount = 0, option;
    do
```



```

{
    printf("\nElectromagnetic Field Calculator\n");
    printf("1. Add Field Point\n");
    printf("2. Display Field Points\n");
    printf("3. Calculate Field Intensity\n");
    printf("4. Exit\n");
    printf("Enter the option: ");
    scanf("%d", &option);
    switch (option)
    {
        case 1: addFieldPoint(&fieldPoints, &pointCount);
                break;
        case 2: displayFieldPoints(fieldPoints, pointCount);
                break;
        case 3: calculateFieldIntensity(fieldPoints, pointCount);
                break;
        case 4: for (int i = 0; i < pointCount; i++)
                    free((int *)fieldPoints[i].pointID);
                    free(fieldPoints);
                    printf("Exiting the system\n");
                    break;
        default: printf("Invalid option\n");
    }
} while (option != 4);
return 0;
}

```

```

void addFieldPoint(struct FieldPoint **fieldPoints, int *pointCount)
{
    int tempPointID;
    struct FieldPoint newFieldPoint;
    printf("Enter Field Point ID: ");
    scanf("%d", &tempPointID);
    int *pointIDPtr = (int *)malloc(sizeof(int));
    *pointIDPtr = tempPointID;
    newFieldPoint.pointID = pointIDPtr;
    printf("Enter Position: ");
    scanf("%f %f %f", &newFieldPoint.position[0], &newFieldPoint.position[1],
                                                    &newFieldPoint.position[2]);
    printf("Enter Field Type (1 for Electric, 2 for Magnetic): ");
    scanf("%d", &newFieldPoint.fieldType);
    switch (newFieldPoint.fieldType)
    {
        case 1: printf("Enter Electric Field Components (Ex Ey Ez): ");
                scanf("%f %f %f", &newFieldPoint.field.electricField[0],
                                    &newFieldPoint.field.electricField[1],
                                    &newFieldPoint.field.electricField[2]);
                break;
        case 2: printf("Enter Magnetic Field Components (Bx By Bz): ");
                scanf("%f %f %f", &newFieldPoint.field.magneticField[0],
                                    &newFieldPoint.field.magneticField[1],
                                    &newFieldPoint.field.magneticField[2]);
                break;
        default: printf("Invalid field type\n");
                return;
    }
}

```

```

    }
    printf("Enter Coordinate System (Cartesian, Cylindrical, Spherical): ");
    scanf("%s", newFieldPoint.coordinateSystem);
    *fieldPoints = (struct FieldPoint *)realloc(*fieldPoints, (*pointCount + 1) *
                                                sizeof(struct FieldPoint));
    (*fieldPoints)[*pointCount] = newFieldPoint;
    (*pointCount)++;
    printf("Field point added successfully\n");
}

```

```

void displayFieldPoints(struct FieldPoint *fieldPoints, int pointCount)
{
    if (pointCount == 0)
    {
        printf("No field points available\n");
        return;
    }
    printf("\nField Points Details\n");
    for (int i = 0; i < pointCount; i++)
    {
        printf("Field Point ID: %d\n", *(fieldPoints[i].pointID));
        printf("Position: (%.2f, %.2f, %.2f)\n", fieldPoints[i].position[0],
            fieldPoints[i].position[1], fieldPoints[i].position[2]);
        printf("Coordinate System: %s\n", fieldPoints[i].coordinateSystem);
        switch (fieldPoints[i].fieldType)
        {
            case 1: printf("Field Type: Electric\n");

```

```

        printf("Field Components: (%.2f, %.2f, %.2f)\n",
               fieldPoints[i].field.electricField[0],
               fieldPoints[i].field.electricField[1],
               fieldPoints[i].field.electricField[2]);

        break;

    case 2: printf("Field Type: Magnetic\n");

        printf("Field Components: (%.2f, %.2f, %.2f)\n",
               fieldPoints[i].field.magneticField[0],
               fieldPoints[i].field.magneticField[1],
               fieldPoints[i].field.magneticField[2]);

        break;

    default: printf("Invalid field type\n");
}
}
}

void calculateFieldIntensity(struct FieldPoint *fieldPoints, int pointCount)
{
    if (pointCount == 0)
    {
        printf("No field points available for intensity calculation\n");
        return;
    }

    printf("\nField Intensity Calculation\n");
    for (int i = 0; i < pointCount; i++)
    {
        float intensity;
        switch (fieldPoints[i].fieldType)
        {

```

```

        case 1: intensity = sqrt(pow(fieldPoints[i].field.electricField[0], 2) +
                                pow(fieldPoints[i].field.electricField[1], 2) +
                                pow(fieldPoints[i].field.electricField[2], 2));
        printf("Field Point ID: %d\n", *(fieldPoints[i].pointID));
        printf("Electric Field Intensity: %.2f\n", intensity);
        break;

        case 2: intensity = sqrt(pow(fieldPoints[i].field.magneticField[0], 2) +
                                pow(fieldPoints[i].field.magneticField[1], 2) +
                                pow(fieldPoints[i].field.magneticField[2], 2));
        printf("Field Point ID: %d\n", *(fieldPoints[i].pointID));
        printf("Magnetic Field Intensity: %.2f\n", intensity);
        break;

        default: printf("Invalid field type for Field Point ID: %d\n",
                        *(fieldPoints[i].pointID));
    }
}
}

```

3. Atomic Energy Level Tracker

Description:

Track the energy levels of atoms and the transitions between them.

Specifications:

- Structure: Contains atomic details (element name, energy levels, and transition probabilities).
- Array: Stores energy levels for different atoms.
- Union: Represents different energy states.
- Strings: Represent element names.

- const Pointers: Protect atomic data.
- Double Pointers: Allocate memory for dynamically adding new elements.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Atom
```

```
{  
    const char *elementName;  
    int energyLevelCount;  
    float *energyLevels;  
    union  
    {  
        float *transitionProbabilities;  
        float *lifetimes;  
    } stateData;  
    int stateType; // 1 for Transition Probabilities, 2 for Lifetimes  
};
```

```
void addAtom(struct Atom **atoms, int *atomCount);
```

```
void displayAtoms(struct Atom *atoms, int atomCount);
```

```
void calculateTransitions(struct Atom *atoms, int atomCount);
```

```
int main()
```

```
{  
    struct Atom *atoms = NULL;
```

```

int atomCount = 0, option;
do
{
    printf("\nAtomic Energy Level Tracker\n");
    printf("1. Add Atom\n");
    printf("2. Display Atoms\n");
    printf("3. Calculate Transitions\n");
    printf("4. Exit\n");
    printf("Enter your option: ");
    scanf("%d", &option);
    switch (option)
    {
        case 1: addAtom(&atoms, &atomCount);
                break;
        case 2: displayAtoms(atoms, atomCount);
                break;
        case 3: calculateTransitions(atoms, atomCount);
                break;
        case 4: for (int i = 0; i < atomCount; i++)
                {
                    free((char *)atoms[i].elementName);
                    free(atoms[i].energyLevels);
                    free(atoms[i].stateData.transitionProbabilities);
                }
                free(atoms);
                printf("Exiting the system\n");
                break;
    }
}

```

```

        default: printf("Invalid option\n");
    }
} while (option != 4);
return 0;
}

```

```

void addAtom(struct Atom **atoms, int *atomCount)
{
    char tempName[50];
    int levels;
    printf("Enter Element Name: ");
    scanf("%s", tempName);
    char *elementNamePtr = (char *)malloc((strlen(tempName) + 1) *
                                           sizeof(char));

    strcpy(elementNamePtr, tempName);
    struct Atom newAtom;
    newAtom.elementName = elementNamePtr;
    printf("Enter Number of Energy Levels: ");
    scanf("%d", &levels);
    newAtom.energyLevelCount = levels;
    newAtom.energyLevels = (float *)malloc(levels * sizeof(float));
    printf("Enter Energy Levels: ");
    for (int i = 0; i < levels; i++)
        scanf("%f", &newAtom.energyLevels[i]);
    printf("Enter State Type (1 for Transition Probabilities, 2 for Lifetimes): ");
    scanf("%d", &newAtom.stateType);
    newAtom.stateData.transitionProbabilities = (float *)malloc(levels *

```



```

                                                                    sizeof(float));

if (newAtom.stateType == 1)
{
    printf("Enter Transition Probabilities: ");
    for (int i = 0; i < levels; i++)
        scanf("%f", &newAtom.stateData.transitionProbabilities[i]);
}
else if (newAtom.stateType == 2)
{
    printf("Enter Lifetimes: ");
    for (int i = 0; i < levels; i++)
        scanf("%f", &newAtom.stateData.lifetimes[i]);
}
else
{
    printf("Invalid state type\n");
    free(elementNamePtr);
    free(newAtom.energyLevels);
    free(newAtom.stateData.transitionProbabilities);
    return;
}

*atoms = (struct Atom *)realloc(*atoms, (*atomCount + 1) * sizeof(struct
                                                                    Atom));

(*atoms)[*atomCount] = newAtom;
(*atomCount)++;
printf("Atom added successfully\n");
}

```

```

void displayAtoms(struct Atom *atoms, int atomCount)
{
    if (atomCount == 0)
    {
        printf("No atoms available\n");
        return;
    }
    printf("\nAtomic Details\n");
    for (int i = 0; i < atomCount; i++)
    {
        printf("Element Name: %s\n", atoms[i].elementName);
        printf("Energy Levels: ");
        for (int j = 0; j < atoms[i].energyLevelCount; j++)
            printf("%.2f ", atoms[i].energyLevels[j]);
        printf("\n");
        if (atoms[i].stateType == 1)
        {
            printf("Transition Probabilities: ");
            for (int j = 0; j < atoms[i].energyLevelCount; j++)
                printf("%.2f ", atoms[i].stateData.transitionProbabilities[j]);
        }
        else if (atoms[i].stateType == 2)
        {
            printf("Lifetimes (s): ");
            for (int j = 0; j < atoms[i].energyLevelCount; j++)
                printf("%.2f ", atoms[i].stateData.lifetimes[j]);
        }
    }
}

```

```

        printf("\n\n");
    }
}

void calculateTransitions(struct Atom *atoms, int atomCount)
{
    if (atomCount == 0)
    {
        printf("No atoms available for transition calculations\n");
        return;
    }
    printf("\nTransition Calculations\n");
    for (int i = 0; i < atomCount; i++)
    {
        printf("Element Name: %s\n", atoms[i].elementName);
        if (atoms[i].stateType == 1)
        {
            for (int j = 0; j < atoms[i].energyLevelCount - 1; j++)
            {
                float transitionEnergy = atoms[i].energyLevels[j + 1] -
                                         atoms[i].energyLevels[j];
                printf("Transition from Level %d to Level %d: %.2f eV (Probability:
                        %.2f)\n", j + 1, j + 2,
                        transitionEnergy, atoms[i].stateData.transitionProbabilities[j]);
            }
        }
        else if (atoms[i].stateType == 2)

```

```

        printf("Lifetimes are provided; no specific transitions calculated\n");
    }
}

```

4. Quantum State Representation System

Description:

Develop a program to represent quantum states and their evolution over time.

Specifications:

- Structure: Holds state properties (wavefunction amplitude, phase, and energy).
- Array: Represents the wavefunction across multiple points.
- Union: Stores amplitude or phase information.
- Strings: Describe state labels (e.g., "ground state," "excited state").
- const Pointers: Protect state properties.
- Double Pointers: Manage quantum states dynamically.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <math.h>
```

```
struct QuantumState
```

```
{
```

```
    const char *stateLabel;
```

```
    int pointCount;
```

```
    float *wavefunction;
```

```
    union
```

```

{
    float *amplitude;
    float *phase;
} stateInfo;
int infoType; // 1 for Amplitude, 2 for Phase
float energy; // Energy of the quantum state (protected)
};

void addQuantumState(struct QuantumState **states, int *stateCount);
void displayQuantumStates(struct QuantumState *states, int stateCount);
void evolveQuantumState(struct QuantumState *states, int stateCount);

int main()
{
    struct QuantumState *states = NULL;
    int stateCount = 0, option;
    do
    {
        printf("\nQuantum State Representation System\n");
        printf("1. Add Quantum State\n");
        printf("2. Display Quantum States\n");
        printf("3. Evolve Quantum States\n");
        printf("4. Exit\n");
        printf("Enter your option: ");
        scanf("%d", &option);
        switch (option)
        {

```

```

    case 1: addQuantumState(&states, &stateCount);
        break;
    case 2: displayQuantumStates(states, stateCount);
        break;
    case 3: evolveQuantumState(states, stateCount);
        break;
    case 4: for (int i = 0; i < stateCount; i++)
        {
            free((char *)states[i].stateLabel);
            free(states[i].wavefunction);
            if (states[i].infoType == 1)
                free(states[i].stateInfo.amplitude);
            else if (states[i].infoType == 2)
                free(states[i].stateInfo.phase);
        }
        free(states);
        printf("Exiting the system\n");
        break;
    default: printf("Invalid option\n");
}
} while (option != 4);
return 0;
}

void addQuantumState(struct QuantumState **states, int *stateCount)
{
    char tempLabel[50];

```

```

int points;
float energy;
printf("Enter State Label: ");
scanf("%s", tempLabel);
char *stateLabelPtr = (char *)malloc((strlen(tempLabel) + 1) * sizeof(char));
if (!stateLabelPtr)
{
    printf("Memory allocation failed for state label\n");
    return;
}
strcpy(stateLabelPtr, tempLabel);
printf("Enter Energy of the State: ");
scanf("%f", &energy);
struct QuantumState newState = {stateLabelPtr, 0, NULL, {NULL}, 0,
                                energy};

printf("Enter Number of Points in the Wavefunction: ");
scanf("%d", &points);
if (points <= 0)
{
    printf("Invalid number of points.\n");
    free(stateLabelPtr);
    return;
}
newState.pointCount = points;
newState.wavefunction = (float *)malloc(points * sizeof(float));
if (!newState.wavefunction)
{

```

```

    printf("Memory allocation failed for wavefunction\n");
    free(stateLabelPtr);
    return;
}

printf("Enter Wavefunction Values: ");
for (int i = 0; i < points; i++)
    scanf("%f", &newState.wavefunction[i]);

printf("Enter Information Type (1 for Amplitude, 2 for Phase): ");
scanf("%d", &newState.infoType);

if (newState.infoType == 1)
{
    newState.stateInfo.amplitude = (float *)malloc(points * sizeof(float));
    if (!newState.stateInfo.amplitude)
    {
        printf("Memory allocation failed for amplitude.\n");
        free(stateLabelPtr);
        free(newState.wavefunction);
        return;
    }
    printf("Enter Amplitude Values: ");
    for (int i = 0; i < points; i++)
        scanf("%f", &newState.stateInfo.amplitude[i]);
}

else if (newState.infoType == 2)
{
    newState.stateInfo.phase = (float *)malloc(points * sizeof(float));
    if (!newState.stateInfo.phase)

```



```

{
    printf("Memory allocation failed for phase.\n");
    free(stateLabelPtr);
    free(newState.wavefunction);
    return;
}

printf("Enter Phase Values: ");
for (int i = 0; i < points; i++)
    scanf("%f", &newState.stateInfo.phase[i]);
}
else
{
    printf("Invalid information type\n");
    free(stateLabelPtr);
    free(newState.wavefunction);
    return;
}

*states = (struct QuantumState *)realloc(*states, (*stateCount + 1) *
                                         sizeof(struct QuantumState));

if (!*states)
{
    printf("Memory allocation failed for states array.\n");
    free(stateLabelPtr);
    free(newState.wavefunction);
    if (newState.infoType == 1)
        free(newState.stateInfo.amplitude);
    else if (newState.infoType == 2)

```

```

        free(newState.stateInfo.phase);
    return;
}
(*states)[*stateCount] = newState;
(*stateCount)++;
printf("Quantum state added successfully\n");
}

```

```

void displayQuantumStates(struct QuantumState *states, int stateCount)
{
    if (stateCount == 0)
    {
        printf("No quantum states available\n");
        return;
    }
    printf("\nQuantum State Details\n");
    for (int i = 0; i < stateCount; i++)
    {
        printf("State Label: %s\n", states[i].stateLabel);
        printf("Energy: %.2f eV\n", states[i].energy);
        printf("Wavefunction Values: ");
        for (int j = 0; j < states[i].pointCount; j++)
            printf("%.2f ", states[i].wavefunction[j]);
        printf("\n");
        if (states[i].infoType == 1)
        {
            printf("Amplitudes: ");

```

```

        for (int j = 0; j < states[i].pointCount; j++)
            printf("%.2f ", states[i].stateInfo.amplitude[j]);
    }
    else if (states[i].infoType == 2)
    {
        printf("Phases (radians): ");
        for (int j = 0; j < states[i].pointCount; j++)
            printf("%.2f ", states[i].stateInfo.phase[j]);
    }
    printf("\n\n");
}
}

```

```

void evolveQuantumState(struct QuantumState *states, int stateCount)
{
    if (stateCount == 0)
    {
        printf("No quantum states available for evolution\n");
        return;
    }
    printf("\nQuantum State Evolution\n");
    for (int i = 0; i < stateCount; i++)
    {
        printf("Evolving State: %s\n", states[i].stateLabel);
        for (int j = 0; j < states[i].pointCount; j++)
        {
            float phaseShift = states[i].energy * 0.01;

```

```

        if (states[i].infoType == 2)
            states[i].stateInfo.phase[j] += phaseShift;
        }
        printf("Evolution completed for state: %s\n", states[i].stateLabel);
    }
}

```

5. Optics Simulation Tool

Description:

Simulate light rays passing through different optical elements.

Specifications:

- Structure: Represents optical properties (refractive index, focal length).
- Array: Stores light ray paths.
- Union: Handles lens or mirror parameters.
- Strings: Represent optical element types.
- const Pointers: Protect optical properties.
- Double Pointers: Manage arrays of optical elements dynamically.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct OpticalElement
```

```
{
```

```
    const char *elementType;
```

```
    float refractiveIndex;
```

```
    union
```

```

{
    float focalLength;
    float curvature;
} parameters;
int paramType; // 1 for focal length, 2 for curvature
};

struct LightRay
{
    float *path;
    int pointCount;
};

void addOpticalElement(struct OpticalElement **elements, int *elementCount);
void displayOpticalElements(struct OpticalElement *elements,
                             int elementCount);
void simulateLightRay(struct LightRay **rays, int *rayCount,
                      struct OpticalElement *elements, int elementCount);

int main()
{
    struct OpticalElement *elements = NULL;
    struct LightRay *rays = NULL;
    int elementCount = 0, rayCount = 0, option;
    do
    {
        printf("\nOptics Simulation Tool\n");
    }

```

```

printf("1. Add Optical Element\n");
printf("2. Display Optical Elements\n");
printf("3. Simulate Light Rays\n");
printf("4. Exit\n");
printf("Enter the option: ");
scanf("%d", &option);
switch (option)
{
    case 1: addOpticalElement(&elements, &elementCount);
            break;
    case 2: displayOpticalElements(elements, elementCount);
            break;
    case 3: simulateLightRay(&rays, &rayCount, elements, elementCount);
            break;
    case 4: for (int i = 0; i < elementCount; i++)
            free((char *)elements[i].elementType);
            free(elements);
            for (int i = 0; i < rayCount; i++)
                free(rays[i].path);
            free(rays);
            printf("Exiting the simulation tool\n");
            break;
    default: printf("Invalid option\n");
}
} while (option != 4);
return 0;
}

```

```

void addOpticalElement(struct OpticalElement **elements, int *elementCount)
{
    char tempType[50];
    float refrIndex, focalLengthOrCurvature;
    printf("Enter Optical Element Type (Lens/Mirror): ");
    scanf("%s", tempType);
    char *elementTypePtr = (char *)malloc((strlen(tempType) + 1) * sizeof(char));
    if (elementTypePtr == NULL)
    {
        printf("Memory allocation failed for element type\n");
        return;
    }
    strcpy(elementTypePtr, tempType);
    printf("Enter Refractive Index (for Lens only, else enter 1): ");
    scanf("%f", &refrIndex);
    struct OpticalElement newElement = {elementTypePtr, refrIndex, {0}, 0};
    if (strcmp(tempType, "Lens") == 0)
    {
        printf("Enter Focal Length of the Lens: ");
        scanf("%f", &focalLengthOrCurvature);
        newElement.parameters.focalLength = focalLengthOrCurvature;
        newElement.paramType = 1;
    }
    else if (strcmp(tempType, "Mirror") == 0)
    {
        printf("Enter Curvature of the Mirror: ");
        scanf("%f", &focalLengthOrCurvature);
    }
}

```

```

        newElement.parameters.curvature = focalLengthOrCurvature;
        newElement.paramType = 2;
    }
else
{
    printf("Invalid optical element type\n");
    free(elementTypePtr);
    return;
}
struct OpticalElement *temp = (struct OpticalElement *)realloc(*elements,
                                                                (*elementCount + 1) * sizeof(struct OpticalElement));
if (temp == NULL)
{
    printf("Memory allocation failed for optical elements.\n");
    free(elementTypePtr);
    return;
}
*elements = temp;
(*elements)[*elementCount] = newElement;
(*elementCount)++;
printf("Optical element added successfully\n");
}

void displayOpticalElements(struct OpticalElement *elements,
                           int elementCount)
{
    if (elementCount == 0)

```



```

{
    printf("No optical elements available\n");
    return;
}
printf("\nOptical Element Details\n");
for (int i = 0; i < elementCount; i++)
{
    printf("Element Type: %s\n", elements[i].elementType);
    printf("Refractive Index: %.2f\n", elements[i].refractiveIndex);
    if (elements[i].paramType == 1)
        printf("Focal Length: %.2f\n", elements[i].parameters.focalLength);
    else if (elements[i].paramType == 2)
        printf("Curvature: %.2f\n", elements[i].parameters.curvature);
    printf("\n");
}
}

void simulateLightRay(struct LightRay **rays, int *rayCount,
                    struct OpticalElement *elements, int elementCount)
{
    if (elementCount == 0)
    {
        printf("No optical elements available for simulation\n");
        return;
    }
    int points;
    printf("Enter Number of Points in the Light Ray Path: ");

```

```

scanf("%d", &points);
struct LightRay newRay;
newRay.pointCount = points;
newRay.path = (float *)malloc(points * 2 * sizeof(float));
if (newRay.path == NULL)
{
    printf("Memory allocation failed for light ray path\n");
    return;
}
printf("Enter Path Points:\n");
for (int i = 0; i < points * 2; i += 2)
{
    printf("Point %d: ", (i / 2) + 1);
    scanf("%f %f", &newRay.path[i], &newRay.path[i + 1]);
}
printf("Simulating light ray through optical elements\n");
for (int i = 0; i < points * 2; i += 2)
{
    newRay.path[i] += 1.0f;
    newRay.path[i + 1] += 1.0f;
}
struct LightRay *temp = (struct LightRay *)realloc(*rays, (*rayCount + 1) *
                                                    sizeof(struct LightRay));
if (temp == NULL)
{
    printf("Memory allocation failed for light rays\n");
    free(newRay.path);
}

```

```

        return;
    }
    *rays = temp;
    (*rays)[*rayCount] = newRay;
    (*rayCount)++;
    printf("Light ray simulation completed\n");
}

```

6. Thermodynamics State Calculator

Description:

Calculate thermodynamic states of a system based on input parameters like pressure, volume, and temperature.

Specifications:

- Structure: Represents thermodynamic properties (P, V, T, and entropy).
- Array: Stores states over a range of conditions.
- Union: Handles dependent properties like energy or entropy.
- Strings: Represent state descriptions.
- const Pointers: Protect thermodynamic data.
- Double Pointers: Allocate state data dynamically for simulation.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct ThermodynamicState
```

```
{
```

```
    char *description;
```

```

float pressure;
float volume;
float temperature;
union
{
    float energy;
    float entropy;
} dependentProperty;
int propertyType; // 1 for Energy, 2 for Entropy
};

void addState(struct ThermodynamicState **states, int *stateCount);
void displayStates(const struct ThermodynamicState *states, int stateCount);

int main()
{
    struct ThermodynamicState *states = NULL;
    int stateCount = 0;
    int option;
    do
    {
        printf("\nThermodynamics State Calculator\n");
        printf("1. Add Thermodynamic State\n");
        printf("2. Display Thermodynamic States\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
    }

```

```

switch (option)
{
    case 1: addState(&states, &stateCount);
            break;
    case 2: displayStates(states, stateCount);
            break;
    case 3: for (int i = 0; i < stateCount; i++)
            free(states[i].description);
            free(states);
            printf("Exiting the program.\n");
            break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addState(struct ThermodynamicState **states, int *stateCount)
{
    char tempDesc[100];
    float pressure, volume, temperature, dependentValue;
    printf("Enter state description: ");
    scanf("%[^\\n]", tempDesc);
    char *descriptionPtr = (char *)malloc((strlen(tempDesc) + 1) * sizeof(char));
    strcpy(descriptionPtr, tempDesc);
    printf("Enter Pressure: ");
    scanf("%f", &pressure);
}

```

```
printf("Enter Volume: ");
scanf("%f", &volume);
printf("Enter Temperature: ");
scanf("%f", &temperature);
printf("Enter Property Type (1 for Energy, 2 for Entropy): ");
int propertyType;
scanf("%d", &propertyType);
if (propertyType == 1)
{
    printf("Enter Internal Energy: ");
    scanf("%f", &dependentValue);
}
else if (propertyType == 2)
{
    printf("Enter Entropy: ");
    scanf("%f", &dependentValue);
}
else
{
    printf("Invalid property type\n");
    free(descriptionPtr);
    return;
}
struct ThermodynamicState newState;
newState.description = descriptionPtr;
newState.pressure = pressure;
newState.volume = volume;
```

```

newState.temperature = temperature;
newState.propertyType = propertyType;
if (propertyType == 1)
    newState.dependentProperty.energy = dependentValue;
else if (propertyType == 2)
    newState.dependentProperty.entropy = dependentValue;
*states = (struct ThermodynamicState *)realloc(*states, (*stateCount + 1) *
                                                sizeof(struct ThermodynamicState));
(*states)[*stateCount] = newState;
(*stateCount)++;
printf("Thermodynamic state added successfully\n");
}

```

```

void displayStates(const struct ThermodynamicState *states, int stateCount)
{
    if (stateCount == 0)
    {
        printf("No thermodynamic states available\n");
        return;
    }
    printf("\nThermodynamic States\n");
    for (int i = 0; i < stateCount; i++)
    {
        printf("State Description: %s\n", states[i].description);
        printf("Pressure: %.2f Pa\n", states[i].pressure);
        printf("Volume: %.2f m^3\n", states[i].volume);
        printf("Temperature: %.2f K\n", states[i].temperature);
    }
}

```

```

    if (states[i].propertyType == 1)
        printf("Internal Energy: %.2f J\n", states[i].dependentProperty.energy);
    else if (states[i].propertyType == 2)
        printf("Entropy: %.2f J/K\n", states[i].dependentProperty.entropy);
    printf("\n");
}
}

```

7. Nuclear Reaction Tracker

Description:

Track the parameters of nuclear reactions like fission and fusion processes.

Specifications:

- Structure: Represents reaction details (reactants, products, energy released).
- Array: Holds data for multiple reactions.
- Union: Represents either energy release or product details.
- Strings: Represent reactant and product names.
- const Pointers: Protect reaction details.
- Double Pointers: Dynamically allocate memory for reaction data.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct NuclearReaction
```

```
{
```

```
    const char *reactants;
```



```

const char *products;
union
{
    float energyReleased;
    const char *productDetails;
} reactionInfo;
int infoType; // 1 for energyReleased, 2 for productDetails
};

void addReaction(struct NuclearReaction **reactions, int *reactionCount);
void displayReactions(const struct NuclearReaction *reactions,
                      int reactionCount);

int main()
{
    struct NuclearReaction *reactions = NULL;
    int reactionCount = 0, option;
    do
    {
        printf("\nNuclear Reaction Tracker\n");
        printf("1. Add Nuclear Reaction\n");
        printf("2. Display Nuclear Reactions\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {

```

```

    case 1: addReaction(&reactions, &reactionCount);
        break;
    case 2: displayReactions(reactions, reactionCount);
        break;
    case 3: for (int i = 0; i < reactionCount; i++)
        {
            free((char *)reactions[i].reactants);
            free((char *)reactions[i].products);
            if (reactions[i].infoType == 2)
                free((char *)reactions[i].reactionInfo.productDetails);
        }
        free(reactions);
        printf("Exiting\n");
        break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addReaction(struct NuclearReaction **reactions, int *reactionCount)
{
    char tempReactants[100], tempProducts[100], tempDetails[100];
    float energy;
    int infoType;
    printf("Enter reactants: ");
    scanf("%s", tempReactants);
}

```

```

char *reactantsPtr = (char *)malloc((strlen(tempReactants) + 1) * sizeof(char));
if (!reactantsPtr)
{
    printf("Memory allocation failed for reactants\n");
    return;
}
strcpy(reactantsPtr, tempReactants);
printf("Enter products: ");
scanf(" %[^\\n]", tempProducts);
char *productsPtr = (char *)malloc((strlen(tempProducts) + 1) * sizeof(char));
if (!productsPtr)
{
    printf("Memory allocation failed for products\n");
    free(reactantsPtr);
    return;
}
strcpy(productsPtr, tempProducts);
printf("Enter information type (1 for Energy Released, 2 for Product Details):
");
if (scanf("%d", &infoType) != 1 || (infoType != 1 && infoType != 2))
{
    printf("Invalid information type\n");
    free(reactantsPtr);
    free(productsPtr);
    while (getchar() != '\\n');
    return;
}
struct NuclearReaction newReaction = {reactantsPtr, productsPtr, {0},

```

```

infoType};

if (infoType == 1)
{
    printf("Enter energy released: ");
    if (scanf("%f", &energy) != 1)
    {
        printf("Invalid input for energy\n");
        free(reactantsPtr);
        free(productsPtr);
        while (getchar() != '\n');
        return;
    }
    newReaction.reactionInfo.energyReleased = energy;
}
else if (infoType == 2)
{
    printf("Enter product details: ");
    scanf(" %[^\n]", tempDetails);
    char *detailsPtr = (char *)malloc((strlen(tempDetails) + 1) * sizeof(char));
    if (!detailsPtr)
    {
        printf("Memory allocation failed for product details.\n");
        free(reactantsPtr);
        free(productsPtr);
        return;
    }
    strcpy(detailsPtr, tempDetails);
}

```

```

        newReaction.reactionInfo.productDetails = detailsPtr;
    }
    struct NuclearReaction *tempReactions = (struct NuclearReaction *)
        realloc(*reactions, (*reactionCount + 1) * sizeof(struct NuclearReaction));
    if (!tempReactions)
    {
        printf("Memory allocation failed\n");
        free(reactantsPtr);
        free(productsPtr);
        if (infoType == 2)
            free((void *)newReaction.reactionInfo.productDetails);
        return;
    }
    *reactions = tempReactions;
    (*reactions)[*reactionCount] = newReaction;
    (*reactionCount)++;
    printf("Nuclear reaction added successfully\n");
}

```

```

void displayReactions(const struct NuclearReaction *reactions,
                    int reactionCount)
{
    if (reactionCount == 0)
    {
        printf("No nuclear reactions available\n");
        return;
    }
}

```

```

printf("\nNuclear Reactions\n");
for (int i = 0; i < reactionCount; i++)
{
    printf("Reactants: %s\n", reactions[i].reactants);
    printf("Products: %s\n", reactions[i].products);
    if (reactions[i].infoType == 1)
        printf("Energy Released: %.2f MeV\n",
               reactions[i].reactionInfo.energyReleased);
    else if (reactions[i].infoType == 2)
        printf("Product Details: %s\n", reactions[i].reactionInfo.productDetails);
    printf("\n");
}
}

```

8. Gravitational Field Simulation

Description:

Simulate the gravitational field of massive objects in a system.

Specifications:

- Structure: Contains object properties (mass, position, field strength).
- Array: Stores field values at different points.
- Union: Handles either mass or field strength as parameters.
- Strings: Represent object labels (e.g., "Planet A," "Star B").
- const Pointers: Protect object properties.
- Double Pointers: Dynamically allocate memory for gravitational field data.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

#include <string.h>
#include <math.h>

#define GRAVITATIONAL_CONSTANT 6.67430e-11

struct GravitationalObject
{
    const char *label;
    union
    {
        double mass;
        double fieldStrength;
    } property;
    double position[3];
    int propertyType; // 1 for mass, 2 for field strength
};

void addObject(struct GravitationalObject **objects, int *objectCount);
void displayObjects(const struct GravitationalObject *objects, int objectCount);
void calculateFieldAtPoint(const struct GravitationalObject *objects,
                           int objectCount);

int main()
{
    struct GravitationalObject *objects = NULL;
    int objectCount = 0, option;
    do

```

```

{
    printf("\nGravitational Field Simulation\n");
    printf("1. Add Gravitational Object\n");
    printf("2. Display Gravitational Objects\n");
    printf("3. Calculate Field at a Point\n");
    printf("4. Exit\n");
    printf("Enter the option: ");
    scanf("%d", &option);
    switch (option)
    {
        case 1: addObject(&objects, &objectCount);
                break;
        case 2: displayObjects(objects, objectCount);
                break;
        case 3: calculateFieldAtPoint(objects, objectCount);
                break;
        case 4: for (int i = 0; i < objectCount; i++)
                    free((char *)objects[i].label);
                free(objects);
                printf("Exiting\n");
                break;
        default: printf("Invalid option\n");
    }
} while (option != 4);
return 0;
}

```



```

void addObject(struct GravitationalObject **objects, int *objectCount)
{
    char tempLabel[100];
    double mass, fieldStrength, position[3];
    int propertyType;
    printf("Enter object label (e.g., Planet A): ");
    scanf("%[^\\n]", tempLabel);
    char *labelPtr = (char *)malloc((strlen(tempLabel) + 1) * sizeof(char));
    if (!labelPtr)
    {
        printf("Memory allocation failed for object label\\n");
        return;
    }
    strcpy(labelPtr, tempLabel);
    printf("Enter property type (1 for Mass, 2 for Field Strength): ");
    if (scanf("%d", &propertyType) != 1 || (propertyType != 1 && propertyType
                                                    != 2))
    {
        printf("Invalid property type\\n");
        free(labelPtr);
        while (getchar() != '\\n');
        return;
    }
    if (propertyType == 1)
    {
        printf("Enter mass: ");
        if (scanf("%lf", &mass) != 1 || mass <= 0)

```

```

    {
        printf("Invalid input for mass\n");
        free(labelPtr);
        while (getchar() != '\n');
        return;
    }
}
else if (propertyType == 2)
{
    printf("Enter field strength: ");
    if (scanf("%lf", &fieldStrength) != 1 || fieldStrength <= 0)
    {
        printf("Invalid input for field strength\n");
        free(labelPtr);
        while (getchar() != '\n');
        return;
    }
}
printf("Enter position (x, y, z): ");
if (scanf("%lf %lf %lf", &position[0], &position[1], &position[2]) != 3)
{
    printf("Invalid input for position\n");
    free(labelPtr);
    while (getchar() != '\n');
    return;
}
struct GravitationalObject newObject = {labelPtr, {0}, {0}, propertyType};

```

```

if (propertyType == 1)
    newObject.property.mass = mass;
else if (propertyType == 2)
    newObject.property.fieldStrength = fieldStrength;
newObject.position[0] = position[0];
newObject.position[1] = position[1];
newObject.position[2] = position[2];
struct GravitationalObject *tempObjects = (struct GravitationalObject *)
    realloc(*objects, (*objectCount + 1) * sizeof(struct GravitationalObject));
if (!tempObjects)
{
    printf("Memory allocation failed\n");
    free(labelPtr);
    return;
}
*objects = tempObjects;
(*objects)[*objectCount] = newObject;
(*objectCount)++;
printf("Gravitational object added successfully\n");
}

```

```

void displayObjects(const struct GravitationalObject *objects, int objectCount)
{
    if (objectCount == 0)
    {
        printf("No gravitational objects available\n");
        return;
    }
}

```

```

    }
    printf("\nGravitational Objects\n");
    for (int i = 0; i < objectCount; i++)
    {
        printf("Label: %s\n", objects[i].label);
        printf("Position: (%.2lf, %.2lf, %.2lf) meters\n", objects[i].position[0],
            objects[i].position[1], objects[i].position[2]);
        if (objects[i].propertyType == 1)
            printf("Mass: %.2e kg\n", objects[i].property.mass);
        else if (objects[i].propertyType == 2)
            printf("Field Strength: %.2e N/kg\n", objects[i].property.fieldStrength);
        printf("\n");
    }
}

void calculateFieldAtPoint(const struct GravitationalObject *objects,
                           int objectCount)
{
    if (objectCount == 0)
    {
        printf("No gravitational objects available for calculation\n");
        return;
    }
    double point[3];
    printf("Enter point coordinates (x, y, z): ");
    if (scanf("%lf %lf %lf", &point[0], &point[1], &point[2]) != 3)
    {

```

```

    printf("Invalid input for point coordinates.\n");
    while (getchar() != '\n');
    return;
}
double totalField = 0.0;
for (int i = 0; i < objectCount; i++)
{
    if (objects[i].propertyType == 1)
    {
        double dx = point[0] - objects[i].position[0];
        double dy = point[1] - objects[i].position[1];
        double dz = point[2] - objects[i].position[2];
        double distance = sqrt(dx * dx + dy * dy + dz * dz);
        if (distance == 0)
        {
            printf("Point coincides with the object '%s'\n", objects[i].label);
            return;
        }
        totalField += GRAVITATIONAL_CONSTANT * objects[i].property.mass
                    / (distance * distance);
    }
}
printf("Total Gravitational Field Strength at Point: %.2e\n", totalField);
}

```

9. Wave Interference Analyzer

Description:

Analyze interference patterns produced by waves from multiple sources.

Specifications:

- Structure: Represents wave properties (amplitude, wavelength, and phase).
- Array: Stores wave interference data at discrete points.
- Union: Handles either amplitude or phase information.
- Strings: Represent wave source labels.
- const Pointers: Protect wave properties.
- Double Pointers: Manage dynamic allocation of wave sources.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Wave
```

```
{  
    const char *sourceLabel;  
    const float *amplitude;  
    const float *wavelength;  
    float phase;  
    union  
    {  
        float amplitudeInfo;  
        float phaseInfo;  
    } waveData;  
    int dataType; // 1 for Amplitude, 2 for Phase  
};
```

```
void addWave(struct Wave **waves, int *waveCount);  
void displayWaveDetails(struct Wave *waves, int waveCount);  
void analyzeInterference(struct Wave *waves, int waveCount, float pointX,  
                                                                    float pointY);
```

```
int main()  
{  
    struct Wave *waves = NULL;  
    int waveCount = 0, option;  
    do  
    {  
        printf("\nWave Interference Analyzer\n");  
        printf("1. Add Wave Source\n");  
        printf("2. Display Wave Details\n");  
        printf("3. Analyze Interference at a Point\n");  
        printf("4. Exit\n");  
        printf("Enter the option: ");  
        scanf("%d", &option);  
        switch (option)  
        {  
            case 1: addWave(&waves, &waveCount); break;  
            case 2: displayWaveDetails(waves, waveCount); break;  
            case 3: if (waveCount == 0)  
                    printf("No waves available for analysis\n");  
                else  
                {  
                    float pointX, pointY;
```

```

        printf("Enter the point coordinates (x y): ");
        scanf("%f %f", &pointX, &pointY);
        analyzeInterference(waves, waveCount, pointX, pointY);
    }
    break;
case 4: for (int i = 0; i < waveCount; i++)
        free((char *)waves[i].sourceLabel);
    free(waves);
    printf("Exiting the analyzer\n");
    break;
default: printf("Invalid option\n");
}
} while (option != 4);
return 0;
}

```

```

void addWave(struct Wave **waves, int *waveCount)
{
    char tempLabel[100];
    struct Wave newWave;
    printf("Enter wave source label: ");
    scanf("%s", tempLabel);
    char *sourceLabelPtr = (char *)malloc(strlen(tempLabel) + 1);
    strcpy(sourceLabelPtr, tempLabel);
    newWave.sourceLabel = sourceLabelPtr;
    const float *amplitudePtr = (float *)malloc(sizeof(float));
    printf("Enter wave amplitude: ");

```



```

scanf("%f", (float *)amplitudePtr);
newWave.amplitude = amplitudePtr;
const float *wavelengthPtr = (float *)malloc(sizeof(float));
printf("Enter wave wavelength: ");
scanf("%f", (float *)wavelengthPtr);
newWave.wavelength = wavelengthPtr;
printf("Enter wave phase: ");
scanf("%f", &newWave.phase);
printf("Enter data type (1 for Amplitude, 2 for Phase): ");
scanf("%d", &newWave.dataType);
switch (newWave.dataType)
{
    case 1: printf("Enter Amplitude Information: ");
            scanf("%f", &newWave.waveData.amplitudeInfo);
            break;
    case 2: printf("Enter Phase Information: ");
            scanf("%f", &newWave.waveData.phaseInfo);
            break;
    default: printf("Invalid data type\n");
            return;
}
*waves = (struct Wave *)realloc(*waves, (*waveCount + 1) *
                                sizeof(struct Wave));
(*waves)[*waveCount] = newWave;
(*waveCount)++;
printf("Wave source added successfully\n");
}

```

```

void displayWaveDetails(struct Wave *waves, int waveCount)
{
    if (waveCount == 0)
    {
        printf("No wave details available\n");
        return;
    }
    printf("\nWave Details\n");
    for (int i = 0; i < waveCount; i++)
    {
        printf("Wave Source: %s\n", waves[i].sourceLabel);
        printf("Amplitude: %.2f\n", *(waves[i].amplitude));
        printf("Wavelength: %.2f\n", *(waves[i].wavelength));
        printf("Phase: %.2f\n", waves[i].phase);
        switch (waves[i].dataType)
        {
            case 1: printf("Amplitude Information: %.2f\n",
                           waves[i].waveData.amplitudeInfo);
                    break;
            case 2: printf("Phase Information: %.2f\n",
                           waves[i].waveData.phaseInfo);
                    break;
            default: printf("Invalid data type\n");
        }
    }
}

```

```

void analyzeInterference(struct Wave *waves, int waveCount, float pointX, float
pointY)
{
    printf("\nAnalyzing interference at point (%.2f, %.2f)\n", pointX, pointY);
    float totalAmplitude = 0;
    float totalPhase = 0;
    for (int i = 0; i < waveCount; i++)
    {
        totalAmplitude += *(waves[i].amplitude);
        totalPhase += waves[i].phase;
    }
    printf("Total Interference Amplitude: %.2f\n", totalAmplitude);
    printf("Total Interference Phase: %.2f\n", totalPhase);
}

```

10. Magnetic Material Property Database

Description:

Create a database to store and retrieve properties of magnetic materials.

Specifications:

- Structure: Represents material properties (permeability, saturation).
- Array: Stores data for multiple materials.
- Union: Handles temperature-dependent properties.
- Strings: Represent material names.
- const Pointers: Protect material data.
- Double Pointers: Allocate material records dynamically.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct MagneticMaterial
```

```
{
```

```
    const char *materialName;
```

```
    const float *permeability;
```

```
    const float *saturation;
```

```
    union
```

```
    {
```

```
        float temperatureCoefficient;
```

```
    } tempProperty;
```

```
    int tempPropertyType; // 1 for Permeability, 2 for Saturation
```

```
};
```

```
void addMaterial(struct MagneticMaterial **materials, int *materialCount);
```

```
void displayMaterialDetails(struct MagneticMaterial *materials,  
                            int materialCount);
```

```
int main()
```

```
{
```

```
    struct MagneticMaterial *materials = NULL;
```

```
    int materialCount = 0, option;
```

```
    do
```

```
    {
```

```
        printf("\nMagnetic Material Property Database\n");
```

```
        printf("1. Add Material\n");
```

```

printf("2. Display Material Details\n");
printf("3. Exit\n");
printf("Enter the option: ");
scanf("%d", &option);
switch (option)
{
    case 1: addMaterial(&materials, &materialCount); break;
    case 2: displayMaterialDetails(materials, materialCount); break;
    case 3: for (int i = 0; i < materialCount; i++)
        {
            free((char *)materials[i].materialName);
            free((float *)materials[i].permeability);
            free((float *)materials[i].saturation);
        }
        free(materials);
        printf("Exiting the database\n");
        break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

void addMaterial(struct MagneticMaterial **materials, int *materialCount)
{
    char tempName[100];
    struct MagneticMaterial newMaterial;

```

```

printf("Enter material name: ");
scanf("%s", tempName);
char *materialNamePtr = (char *)malloc(strlen(tempName) + 1);
strcpy(materialNamePtr, tempName);
newMaterial.materialName = materialNamePtr;
const float *permeabilityPtr = (float *)malloc(sizeof(float));
printf("Enter permeability: ");
scanf("%f", (float *)permeabilityPtr);
newMaterial.permeability = permeabilityPtr;
const float *saturationPtr = (float *)malloc(sizeof(float));
printf("Enter saturation: ");
scanf("%f", (float *)saturationPtr);
newMaterial.saturation = saturationPtr;
printf("Enter temperature-dependent property type (1 for Permeability, 2 for
                                             Saturation): ");
scanf("%d", &newMaterial.tempPropertyType);
switch (newMaterial.tempPropertyType)
{
    case 1: printf("Enter temperature coefficient for permeability: ");
            scanf("%f", &newMaterial.tempProperty.temperatureCoefficient);
            break;
    case 2: printf("Enter temperature coefficient for saturation: ");
            scanf("%f", &newMaterial.tempProperty.temperatureCoefficient);
            break;
    default: printf("Invalid property type\n");
            return;
}

```

```

*materials = (struct MagneticMaterial *)realloc(*materials, (*materialCount +
                                                    1) * sizeof(struct MagneticMaterial));
(*materials)[*materialCount] = newMaterial;
(*materialCount)++;
printf("Material added successfully\n");
}

```

```

void displayMaterialDetails(struct MagneticMaterial *materials,
                           int materialCount)
{
    if (materialCount == 0)
    {
        printf("No materials available\n");
        return;
    }
    printf("\nMagnetic Material Details\n");
    for (int i = 0; i < materialCount; i++) {
        printf("Material Name: %s\n", materials[i].materialName);
        printf("Permeability: %.2f\n", *(materials[i].permeability));
        printf("Saturation: %.2f\n", *(materials[i].saturation));
        switch (materials[i].tempPropertyType)
        {
            case 1: printf("Temperature Coefficient for Permeability: %.2f\n",
                           materials[i].tempProperty.temperatureCoefficient);
                    break;
            case 2: printf("Temperature Coefficient for Saturation: %.2f\n",
                           materials[i].tempProperty.temperatureCoefficient);

```

```

        break;
    default: printf("Invalid temperature coefficient type\n");
    }
}
}

```

11. Plasma Dynamics Simulator

Description:

Simulate the behavior of plasma under various conditions.

Specifications:

- Structure: Represents plasma parameters (density, temperature, and electric field).
- Array: Stores simulation results.
- Union: Handles either density or temperature data.
- Strings: Represent plasma types.
- const Pointers: Protect plasma parameters.
- Double Pointers: Manage dynamic allocation for simulation data.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Plasma
```

```
{
```

```
    const char *plasmaType;
```

```
    const float *electricField;
```

```
    union
```



```

{
    float density;
    float temperature;
} plasmaProperty;
int propertyType; // 1 for Density, 2 for Temperature
};

void addPlasmaSimulation(struct Plasma **simulations, int *simulationCount);
void displaySimulations(struct Plasma *simulations, int simulationCount);

int main()
{
    struct Plasma *simulations = NULL;
    int simulationCount = 0, option;
    do
    {
        printf("\nPlasma Dynamics Simulator\n");
        printf("1. Add Plasma Simulation\n");
        printf("2. Display Simulation Results\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addPlasmaSimulation(&simulations, &simulationCount);
                    break;
            case 2: displaySimulations(simulations, simulationCount);

```

```

        break;
    case 3: for (int i = 0; i < simulationCount; i++)
    {
        free((char *)simulations[i].plasmaType);
        free((float *)simulations[i].electricField);
    }
    free(simulations);
    printf("Exiting the simulator\n");
    break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addPlasmaSimulation(struct Plasma **simulations, int *simulationCount)
{
    char tempType[100];
    struct Plasma newSimulation;
    printf("Enter plasma type: ");
    scanf("%s", tempType);
    char *plasmaTypePtr = (char *)malloc(strlen(tempType) + 1);
    strcpy(plasmaTypePtr, tempType);
    newSimulation.plasmaType = plasmaTypePtr;
    const float *electricFieldPtr = (float *)malloc(sizeof(float));
    printf("Enter electric field strength: ");
    scanf("%f", (float *)electricFieldPtr);
}

```

```

newSimulation.electricField = electricFieldPtr;
printf("Enter property type (1 for Density, 2 for Temperature): ");
scanf("%d", &newSimulation.propertyType);
switch (newSimulation.propertyType)
{
    case 1: printf("Enter plasma density: ");
            scanf("%f", &newSimulation.plasmaProperty.density);
            break;
    case 2: printf("Enter plasma temperature: ");
            scanf("%f", &newSimulation.plasmaProperty.temperature);
            break;
    default: printf("Invalid property type\n");
            return;
}
*simulations = (struct Plasma *)realloc(*simulations, (*simulationCount + 1)
                                         * sizeof(struct Plasma));
(*simulations)[*simulationCount] = newSimulation;
(*simulationCount)++;
printf("Plasma simulation added successfully\n");
}

void displaySimulations(struct Plasma *simulations, int simulationCount)
{
    if (simulationCount == 0)
    {
        printf("No simulation data available\n");
        return;
    }
}

```

```

}
printf("\nPlasma Simulation Results\n");
for (int i = 0; i < simulationCount; i++)
{
    printf("Plasma Type: %s\n", simulations[i].plasmaType);
    printf("Electric Field Strength: %.2f\n", *(simulations[i].electricField));
    if (simulations[i].propertyType == 1)
        printf("Plasma Density: %.2f\n", simulations[i].plasmaProperty.density);
    else if (simulations[i].propertyType == 2)
        printf("Plasma Temperature: %.2f\n",
               simulations[i].plasmaProperty.temperature);
    else
        printf("Invalid property type\n");
}
}

```

12. Kinematics Equation Solver

Description:

Solve complex kinematics problems for objects in motion.

Specifications:

- Structure: Represents object properties (initial velocity, acceleration, displacement).
- Array: Stores time-dependent motion data.
- Union: Handles either velocity or displacement equations.
- Strings: Represent motion descriptions.
- const Pointers: Protect object properties.
- Double Pointers: Dynamically allocate memory for motion data.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct Motion
{
    const char *description;
    const float *initialVelocity;
    const float *acceleration;
    union
    {
        float velocity;
        float displacement;
    } equationResult;
    int equationType; // 1 for Velocity, 2 for Displacement
};
```

```
void addMotionData(struct Motion **motions, int *motionCount);
void displayMotionData(struct Motion *motions, int motionCount);
void freeMemory(struct Motion *motions, int motionCount);
```

```
int main()
{
    struct Motion *motions = NULL;
    int motionCount = 0, option;
    do
    {
```

```

printf("\nKinematics Equation Solver\n");
printf("1. Add Motion Data\n");
printf("2. Display Motion Data\n");
printf("3. Exit\n");
printf("Enter the option: ");
scanf("%d", &option);
switch (option)
{
    case 1: addMotionData(&motions, &motionCount);
            break;
    case 2: displayMotionData(motions, motionCount);
            break;
    case 3: freeMemory(motions, motionCount);
            printf("Exiting the solver\n");
            break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

void addMotionData(struct Motion **motions, int *motionCount)
{
    char tempDescription[100];
    struct Motion newMotion;
    printf("Enter motion description: ");
    scanf(" %[^\n]", tempDescription);

```

```

char *descriptionPtr = (char *)malloc(strlen(tempDescription) + 1);
strcpy(descriptionPtr, tempDescription);
newMotion.description = descriptionPtr;
const float *initialVelocityPtr = (float *)malloc(sizeof(float));
printf("Enter initial velocity: ");
scanf("%f", (float *)initialVelocityPtr);
newMotion.initialVelocity = initialVelocityPtr;
const float *accelerationPtr = (float *)malloc(sizeof(float));
printf("Enter acceleration: ");
scanf("%f", (float *)accelerationPtr);
newMotion.acceleration = accelerationPtr;
printf("Enter equation type (1 for Velocity, 2 for Displacement): ");
scanf("%d", &newMotion.equationType);
float time;
printf("Enter time: ");
scanf("%f", &time);
switch (newMotion.equationType)
{
    case 1: newMotion.equationResult.velocity = *newMotion.initialVelocity +
        (*newMotion.acceleration * time);
        break;
    case 2: newMotion.equationResult.displacement =
        (*newMotion.initialVelocity * time) +
        (0.5 * (*newMotion.acceleration) * time * time);
        break;
    default: printf("Invalid equation type\n");
        return;
}

```

```

    }
    *motions = (struct Motion *)realloc(*motions, (*motionCount + 1) *
                                         sizeof(struct Motion));

    (*motions)[*motionCount] = newMotion;
    (*motionCount)++;
    printf("Motion data added successfully\n");
}

```

```

void displayMotionData(struct Motion *motions, int motionCount)
{
    if (motionCount == 0)
    {
        printf("No motion data available\n");
        return;
    }
    printf("\nMotion Data Results\n");
    for (int i = 0; i < motionCount; i++)
    {
        printf("Motion Description: %s\n", motions[i].description);
        printf("Initial Velocity: %.2f m/s\n", *motions[i].initialVelocity);
        printf("Acceleration: %.2f m/s^2\n", *motions[i].acceleration);
        if (motions[i].equationType == 1)
            printf("Velocity after time: %.2f m/s\n",
                   motions[i].equationResult.velocity);
        else if (motions[i].equationType == 2)
            printf("Displacement after time: %.2f m\n",
                   motions[i].equationResult.displacement);
    }
}

```



```

        else
            printf("Invalid equation type\n");
        printf("\n");
    }
}

void freeMemory(struct Motion *motions, int motionCount)
{
    for (int i = 0; i < motionCount; i++)
    {
        free((char *)motions[i].description);
        free((float *)motions[i].initialVelocity);
        free((float *)motions[i].acceleration);
    }
    free(motions);
}

```

13. Spectral Line Database

Description:

Develop a database to store and analyze spectral lines of elements.

Specifications:

- Structure: Represents line properties (wavelength, intensity, and element).
- Array: Stores spectral line data.
- Union: Handles either intensity or wavelength information.
- Strings: Represent element names.
- const Pointers: Protect spectral line data.
- Double Pointers: Allocate spectral line records dynamically.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct SpectralLine
{
    const char *element;
    union
    {
        const float *wavelength;
        const float *intensity;
    } property;
    int propertyType; // 1 for Wavelength, 2 for Intensity
};
```

```
void addSpectralLine(struct SpectralLine **lines, int *lineCount);
void displaySpectralLines(struct SpectralLine *lines, int lineCount);
void freeMemory(struct SpectralLine *lines, int lineCount);
```

```
int main()
{
    struct SpectralLine *lines = NULL;
    int lineCount = 0, option;
    do
    {
        printf("\nSpectral Line Database\n");
        printf("1. Add Spectral Line\n");
```

```

printf("2. Display Spectral Lines\n");
printf("3. Exit\n");
printf("Enter your option: ");
scanf("%d", &option);
switch (option)
{
    case 1: addSpectralLine(&lines, &lineCount);
            break;
    case 2: displaySpectralLines(lines, lineCount);
            break;
    case 3: freeMemory(lines, lineCount);
            printf("Exiting the database\n");
            break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addSpectralLine(struct SpectralLine **lines, int *lineCount)
{
    char tempElement[100];
    struct SpectralLine newLine;
    printf("Enter element name: ");
    scanf(" %s", tempElement);
    char *elementPtr = (char *)malloc(strlen(tempElement) + 1);
    strcpy(elementPtr, tempElement);
}

```

```

newLine.element = elementPtr;
printf("Enter property type (1 for Wavelength, 2 for Intensity): ");
scanf("%d", &newLine.propertyType);
if (newLine.propertyType == 1)
{
    const float *wavelengthPtr = (float *)malloc(sizeof(float));
    printf("Enter wavelength: ");
    scanf("%f", (float *)wavelengthPtr);
    newLine.property.wavelength = wavelengthPtr;
}
else if (newLine.propertyType == 2)
{
    const float *intensityPtr = (float *)malloc(sizeof(float));
    printf("Enter intensity: ");
    scanf("%f", (float *)intensityPtr);
    newLine.property.intensity = intensityPtr;
}
else
{
    printf("Invalid property type\n");
    return;
}
*lines = (struct SpectralLine *)realloc(*lines, (*lineCount + 1) * sizeof(struct
                                                                    SpectralLine));

(*lines)[*lineCount] = newLine;
(*lineCount)++;
printf("Spectral line added successfully\n");

```

```
}
```

```
void displaySpectralLines(struct SpectralLine *lines, int lineCount)
```

```
{
```

```
    if (lineCount == 0)
```

```
    {
```

```
        printf("No spectral line data available\n");
```

```
        return;
```

```
    }
```

```
    printf("\nSpectral Line Data\n");
```

```
    for (int i = 0; i < lineCount; i++)
```

```
    {
```

```
        printf("Element: %s\n", lines[i].element);
```

```
        if (lines[i].propertyType == 1)
```

```
            printf("Wavelength: %.2f nm\n", *lines[i].property.wavelength);
```

```
        else if (lines[i].propertyType == 2)
```

```
            printf("Intensity: %.2f\n", *lines[i].property.intensity);
```

```
        else
```

```
            printf("Invalid property type\n");
```

```
        printf("\n");
```

```
    }
```

```
}
```

```
void freeMemory(struct SpectralLine *lines, int lineCount)
```

```
{
```

```
    for (int i = 0; i < lineCount; i++)
```

```
    {
```

```

    free((char *)lines[i].element);
    if (lines[i].propertyType == 1)
        free((float *)lines[i].property.wavelength);
    else if (lines[i].propertyType == 2)
        free((float *)lines[i].property.intensity);
}
free(lines);
}

```

14. Projectile Motion Simulator

Description:

Simulate and analyze projectile motion under varying conditions.

Specifications:

- Structure: Stores projectile properties (mass, velocity, and angle).
- Array: Stores motion trajectory data.
- Union: Handles either velocity or displacement parameters.
- Strings: Represent trajectory descriptions.
- const Pointers: Protect projectile properties.
- Double Pointers: Manage trajectory records dynamically.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#include <string.h>
```

```
#define GRAVITY 9.8
```

```
struct Projectile
```

```
{  
    const float *mass;  
    const float *velocity;  
    const float *angle;  
};
```

```
struct Trajectory
```

```
{  
    float time;  
    union  
    {  
        float velocity;  
        float displacement;  
    } parameter;  
    int parameterType; // 1 for Velocity, 2 for Displacement  
    char *description;  
};
```

```
void addProjectile(struct Projectile *projectile);
```

```
void simulateMotion(struct Projectile *projectile, struct Trajectory **trajectories,  
                    int *trajectoryCount);
```

```
void displayTrajectory(struct Trajectory *trajectories, int trajectoryCount);
```

```
void freeMemory(struct Projectile *projectile, struct Trajectory *trajectories,  
                int trajectoryCount);
```

```
int main()
```



```

        break;
    default: printf("Invalid option\n");
    }
} while (option != 4);
return 0;
}

```

```

void addProjectile(struct Projectile *projectile)
{
    float *mass = (float *)malloc(sizeof(float));
    float *velocity = (float *)malloc(sizeof(float));
    float *angle = (float *)malloc(sizeof(float));
    printf("Enter mass of the projectile: ");
    scanf("%f", mass);
    printf("Enter initial velocity of the projectile: ");
    scanf("%f", velocity);
    printf("Enter launch angle of the projectile: ");
    scanf("%f", angle);
    projectile->mass = mass;
    projectile->velocity = velocity;
    projectile->angle = angle;
    printf("Projectile added successfully\n");
}

```

```

void simulateMotion(struct Projectile *projectile, struct Trajectory **trajectories,
                    int *trajectoryCount)
{

```

```

float timeStep, totalTime, angleRad, initialVelocityX, initialVelocityY;
int steps;
printf("Enter time step for simulation: ");
scanf("%f", &timeStep);
angleRad = (*projectile->angle) * (M_PI / 180.0);
initialVelocityX = (*projectile->velocity) * cos(angleRad);
initialVelocityY = (*projectile->velocity) * sin(angleRad);
totalTime = (2 * initialVelocityY) / GRAVITY;
steps = (int)(totalTime / timeStep) + 1;
*trajectories = (struct Trajectory *)realloc(*trajectories, steps *
                                           sizeof(struct Trajectory));

*trajectoryCount = steps;
printf("Simulating motion\n");
for (int i = 0; i < steps; i++)
{
    float t = i * timeStep;
    float x = initialVelocityX * t;
    float y = initialVelocityY * t - 0.5 * GRAVITY * t * t;
    float currentVelocity = sqrt(initialVelocityX * initialVelocityX +
                                   pow(initialVelocityY - GRAVITY * t, 2));

    if (y < 0) y = 0;
    (*trajectories)[i].time = t;
    (*trajectories)[i].parameterType = 2;
    (*trajectories)[i].parameter.displacement = sqrt(x * x + y * y);
    char description[100];
    snprintf(description, sizeof(description), "At time %.2fs:
            displacement=%.2fm", t, (*trajectories)[i].parameter.displacement);

```

```

        (*trajectories)[i].description = (char *)malloc(strlen(description) + 1);
        strcpy((*trajectories)[i].description, description);
        printf("Step %d - %s\n", i + 1, (*trajectories)[i].description);
    }
    printf("Motion simulation completed\n");
}

```

```

void displayTrajectory(struct Trajectory *trajectories, int trajectoryCount)
{
    if (trajectoryCount == 0)
    {
        printf("No trajectory data available\n");
        return;
    }
    printf("\nTrajectory Data:\n");
    for (int i = 0; i < trajectoryCount; i++)
        printf("%s\n", trajectories[i].description);
}

```

```

void freeMemory(struct Projectile *projectile, struct Trajectory *trajectories,
                int trajectoryCount)
{
    if (projectile->mass) free((float *)projectile->mass);
    if (projectile->velocity) free((float *)projectile->velocity);
    if (projectile->angle) free((float *)projectile->angle);
    for (int i = 0; i < trajectoryCount; i++)
        free(trajectories[i].description);
}

```

```
    free(trajectories);  
}
```

15. Material Stress-Strain Analyzer

Description:

Analyze the stress-strain behavior of materials under different loads.

Specifications:

- Structure: Represents material properties (stress, strain, modulus).
- Array: Stores stress-strain data.
- Union: Handles dependent properties like yield stress or elastic modulus.
- Strings: Represent material names.
- const Pointers: Protect material properties.
- Double Pointers: Allocate stress-strain data dynamically.

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>
```

```
struct Material  
{  
    const char *name;  
    const float *modulus;  
    union  
    {  
        float yieldStress;  
        float elasticModulus;  
    } property;
```

```
};
```

```
struct StressStrainData
```

```
{
```

```
    float stress;
```

```
    float strain;
```

```
    char *description;
```

```
};
```

```
void addMaterial(struct Material *material);
```

```
void addStressStrainData(struct StressStrainData **data, int *dataCount);
```

```
void displayMaterialProperties(const struct Material *material);
```

```
void displayStressStrainData(const struct StressStrainData *data, int dataCount);
```

```
void freeMemory(struct Material *material, struct StressStrainData *data,  
                int dataCount);
```

```
int main()
```

```
{
```

```
    struct Material material = {NULL, NULL};
```

```
    struct StressStrainData *data = NULL;
```

```
    int dataCount = 0, option;
```

```
    do
```

```
    {
```

```
        printf("\nMaterial Stress-Strain Analyzer\n");
```

```
        printf("1. Add Material\n");
```

```
        printf("2. Add Stress-Strain Data\n");
```

```
        printf("3. Display Material Properties\n");
```

```

printf("4. Display Stress-Strain Data\n");
printf("5. Exit\n");
printf("Enter the option: ");
scanf("%d", &option);
switch (option)
{
    case 1: addMaterial(&material);
            break;
    case 2: if (material.name == NULL || material.modulus == NULL)
            printf("Material properties not set\n");
            else
                addStressStrainData(&data, &dataCount);
            break;
    case 3: displayMaterialProperties(&material);
            break;
    case 4: displayStressStrainData(data, dataCount);
            break;
    case 5: freeMemory(&material, data, dataCount);
            printf("Exiting the analyzer\n");
            break;
    default: printf("Invalid option\n");
}
} while (option != 5);
return 0;
}

void addMaterial(struct Material *material)

```

```
{
    char tempName[50];
    float *modulus = (float *)malloc(sizeof(float));
    printf("Enter the name of the material: ");
    scanf("%49s", tempName);
    material->name = strdup(tempName);
    printf("Enter the modulus of elasticity: ");
    scanf("%f", modulus);
    material->modulus = modulus;
    printf("Enter the yield stress of the material: ");
    scanf("%f", &material->property.yieldStress);
    printf("Material added successfully\n");
}
```

[illegible]

```
    (*data)[*dataCount] = newData;
    (*dataCount)++;
    printf("Stress-strain data point added successfully\n");
}
```

```
void displayMaterialProperties(const struct Material *material)
{
    if (material->name == NULL || material->modulus == NULL)
    {
        printf("No material properties available\n");
        return;
    }
    printf("\nMaterial Properties\n");
    printf("Name: %s\n", material->name);
    printf("Modulus of Elasticity: %.2f \n", *(material->modulus));
    printf("Yield Stress: %.2f \n", material->property.yieldStress);
}
```

```
void displayStressStrainData(const struct StressStrainData *data, int dataCount)
{
    if (dataCount == 0)
    {
        printf("No stress-strain data available\n");
        return;
    }
    printf("\nStress-Strain Data\n");
    for (int i = 0; i < dataCount; i++)
```



```

{
    printf("Data Point %d:\n", i + 1);
    printf("Stress: %.2f MPa\n", data[i].stress);
    printf("Strain: %.6f\n", data[i].strain);
    printf("Description: %s\n", data[i].description);
}
}

```

```

void freeMemory(struct Material *material, struct StressStrainData *data,
                int dataCount)

```

```

{
    if (material->name) free((char *)material->name);
    if (material->modulus) free((float *)material->modulus);
    for (int i = 0; i < dataCount; i++)
        free(data[i].description);
    free(data);
}

```